



MÓDULO 2 · T05

Listas y Navegación con FlatList

 *Listas de alto rendimiento*

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

¿Qué aprenderás hoy?

En esta sesión dominarás el uso de `FlatList` para renderizar listas largas de forma eficiente en React Native. Al terminar, serás capaz de construir listas de alto rendimiento con separadores, estados vacíos y cuadrículas.

1

FlatList vs ScrollView

Entender por qué `FlatList` renderiza solo los ítems visibles y cuándo usarla.

2

renderItem y keyExtractor

Definir cómo se renderiza cada ítem y extraer claves únicas e inmutables.

3

Componentes auxiliares

Usar `ItemSeparatorComponent`, `ListEmptyComponent` y `ListHeaderComponent`.

4

Cuadrícula con numColumns

Transformar una lista en una cuadrícula con `numColumns={2}` y ajustar el layout.

¿Por qué FlatList?

La diferencia fundamental entre `ScrollView` y `FlatList` está en cómo se gestionan los ítems en memoria. Con listas largas esto marca una enorme diferencia de rendimiento.

✗ `ScrollView`

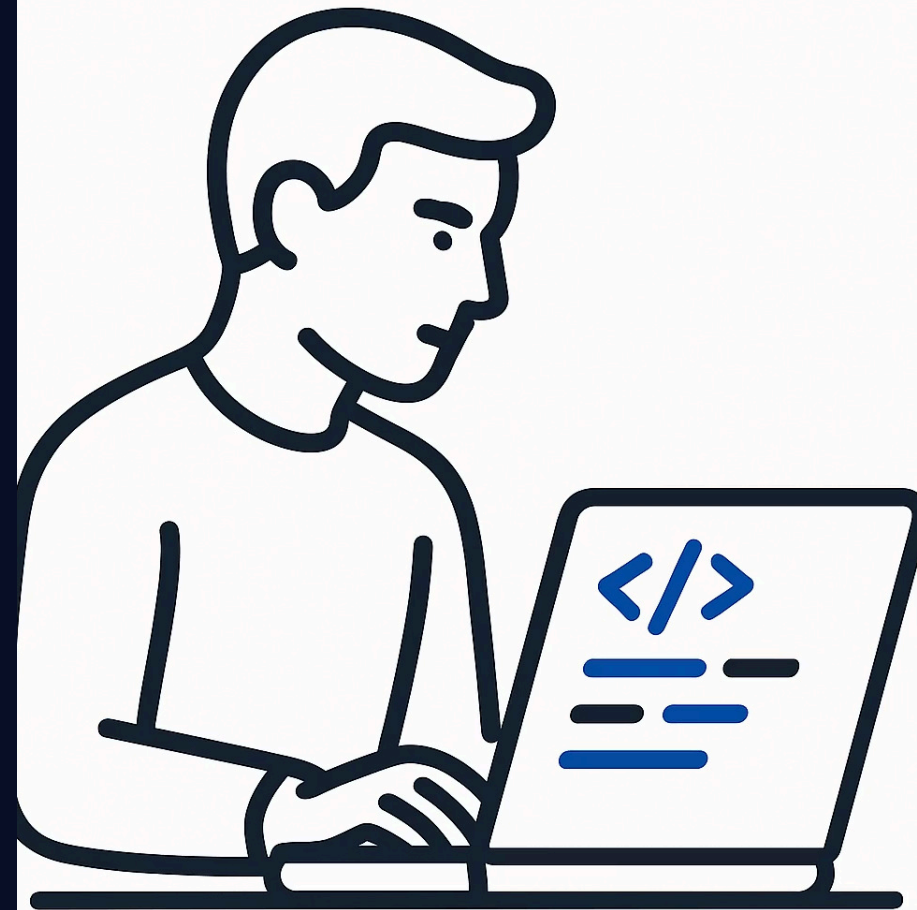
```
<ScrollView>
  {items.map(item => (
    <Row key={item.id} item={item}
  />
  ))}
</ScrollView>
```

Renderiza **todos** los ítems a la vez, aunque no sean visibles en pantalla.

✓ `FlatList`

```
<FlatList
  data={items}
  renderItem={({ item }) => <Row
    item={item} />}
  keyExtractor={(item) =>
    item.id.toString()}
/>
```

Renderiza solo los ítems **visibles** – virtualización automática.



❗ Usa `ScrollView` únicamente para contenido estático y corto. Para cualquier lista de datos, `FlatList` es siempre la opción correcta.

Anatomía de FlatList

Estas son las props fundamentales que necesitas conocer para construir cualquier lista en React Native. Cada una tiene una responsabilidad muy clara y bien delimitada.

Ejemplo completo

```
<FlatList
  data={pokemons}                // array de datos
  renderItem={({ item, index }) => ( // cómo renderizar cada item
    <PokemonListItem pokemon={item} />
  )}
  keyExtractor={(item) => item.id.toString()} // clave única
  ItemSeparatorComponent={() => <View style={{ height: 8 }} />}
  ListEmptyComponent={<Text>No hay Pokémon</Text>}
  ListHeaderComponent={<Text style={styles.header}>Pokédex</Text>}
  contentContainerStyle={{ padding: 12 }}
/>
```

Props clave

data

El array de ítems a renderizar.

renderItem

Función `{ item, index } => JSX` por cada ítem.

keyExtractor

Genera una string única e inmutable por ítem.

ItemSeparatorComponent

Componente renderizado entre cada ítem.

keyExtractor — claves estables

La `keyExtractor` es crítica para el rendimiento y la corrección visual de la lista. React Native la usa internamente para rastrear qué ítems han cambiado, se han añadido o eliminado entre renders.

✓ Correcto: ID único e inmutable

```
keyExtractor={(item) => item.id.toString()}
```

El id del ítem nunca cambia, identifica de forma unívoca cada elemento del array.

✗ Evitar: índice de array

```
keyExtractor={(_, index) => index.toString()}
```

El índice cambia al reordenar, insertar o eliminar ítems – provoca re-renders innecesarios o bugs visuales.

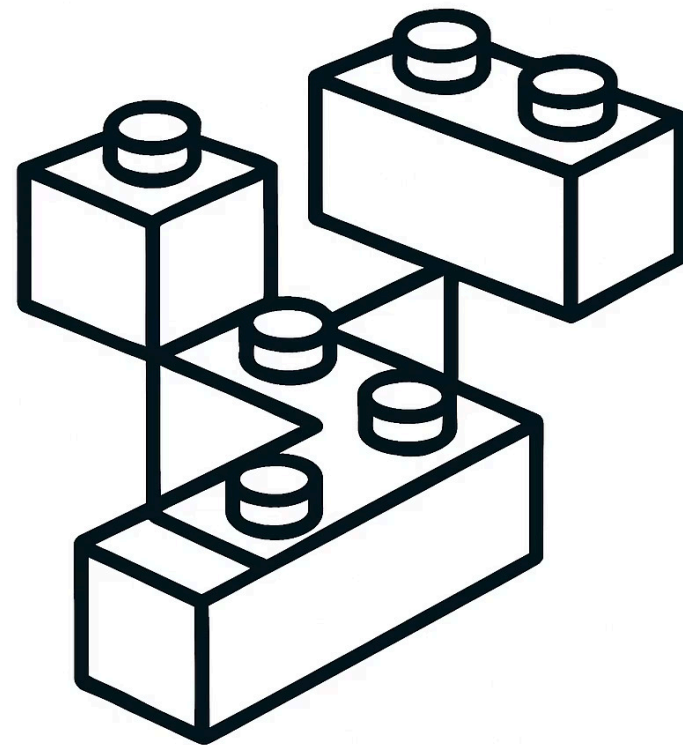
⚠ Una key inestable provoca re-renders innecesarios o bugs visuales difíciles de depurar. Usa siempre el `id` del dato.

numColumns — cuadrícula

La prop `numColumns` transforma una lista vertical en una cuadrícula. Cada ítem debe usar `flex: 1` para repartir el espacio equitativamente entre columnas.

```
<FlatList
  data={pokemons}
  numColumns={2}
  renderItem={({ item }) => (
    <View style={{ flex: 1, margin: 4 }}>
      <PokemonCard pokemon={item} />
    </View>
  )}
  keyExtractor={(item) => item.id.toString()}
/>
```

- ❗ Si cambias `numColumns` en tiempo de ejecución, debes cambiar también la `key` del `FlatList` para forzar un remontaje completo y evitar errores de layout.



PokeApp — PokemonList.tsx

Este es el componente completo tal y como aparece en la aplicación de ejemplo. Observa cómo `FlatList` queda completamente encapsulado dentro del componente, exponiendo solo las props `data` y `onSelect` al exterior.

```
export default function PokemonList({
  data,
  onSelect
}: Props) {
  return (
    <FlatList
      data={data}
      renderItem={({ item }) => (
        <PokemonListItem
          pokemon={item}
          onSelect={onSelect}
        />
      )}
      keyExtractor={(item) => item.id.toString()}
      contentContainerStyle={styles.list}
    />
  );
}
```

📌 **Buena práctica:** encapsula la `FlatList` en su propio componente y expón solo las props necesarias. Así cambiar la implementación interna no afecta al resto de la app.

PokemonListItem — barra de color dinámica

Cada ítem de la lista aplica un color dinámico según el tipo principal del Pokémon, consultando el diccionario `tipoColor`. El color se usa como acento visual en una franja lateral.

Cálculo del color

```
const tipoColor: Record<TipoPokemon, string> = {  
  normal: '#A8A878', fuego: '#F08030', agua: '#6890F0', planta:  
    '#78C850',  
  electrico: '#F8D030', hielo: '#98D8D8', lucha: '#C03028', veneno:  
    '#A040A0',  
  tierra: '#E0C068', volador: '#A890F0', psiquico: '#F85888', bicho:  
    '#A8B820',  
  roca: '#B8A038', fantasma: '#705898', dragon: '#7038F8',  
  siniestro: '#705848',  
  acero: '#B8B8D0', hada: '#EE99AC',  
};
```

Se asigna un **color por tipo**.

Estructura del ítem

```
const primaryColor = tipoColor[tipos[0]];  
  
<TouchableOpacity style={s.listCard} onPress={handlePress}>  
  <View style={[styles.colorStripe, { backgroundColor:  
    primaryColor }} />  
  <Image source={{ uri: thumbnail }} style={styles.image} />  
  <View style={styles.info}>  
    <Text style={s.listCardIndex}>#{String(indice).padStart(3, '0')}</Text>  
    <Text style={s.listCardName}>{nombre}</Text>  
    <PokemonTypeBadgeList tipos={tipos} />  
  </View>  
</TouchableOpacity>
```


Tour starter/PokemonList.tsx

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene la estructura básica con dos puntos marcados como `TODO` que deberás completar durante el ejercicio. Por ahora solo renderiza el nombre del Pokémon y falta el separador.

```
export default function PokemonList({
  data = MOCK_POKEMONS,
  onSelect
}: Props) {
  return (
    <FlatList
      data={data}
      renderItem={({ item }) => (
        // TODO: renderiza cada Pokémon
        // con imagen + nombre
        <View style={styles.item}>
          <Text>{item.nombre}</Text>
        </View>
      )}
      keyExtractor={(item) => item.id.toString()}
      // TODO: ItemSeparatorComponent
    />
  );
}
```

⚠ No modifiques la estructura del `FlatList` ni las props externas – céntrate en los dos `TODO` marcados arriba.

Ejercicio T05

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para afianzar el layout en cuadrícula.

1 Abre el fichero de inicio

Navega a `starter/PokemonList.tsx` en tu editor.

2 Mejora renderItem

Añade la imagen del Pokémon junto al nombre en cada ítem de la lista.

3 Añade ItemSeparatorComponent

Inserta un separador con `8px` de espacio vertical entre cada ítem.

4 Añade ListEmptyComponent

Muestra el mensaje `"No hay Pokémon"` cuando el array `data` esté vacío.

5 Bonus: cuadrícula con numColumns

Cambia a `numColumns={2}` y ajusta el layout del ítem con `flex: 1`.

Resumen de la sesión

Estas son las props clave de `FlatList` que has trabajado hoy. Dominarlas te permitirá construir cualquier lista de datos eficiente y bien estructurada en React Native.

Prop	Valor / Tipo	Para qué sirve
<code>data</code>	Array de ítems	Fuente de datos de la lista
<code>renderItem</code>	<code>{ item } => JSX</code>	Función que renderiza cada ítem
<code>keyExtractor</code>	<code>(item) => string</code>	Clave única e inmutable por ítem
<code>ItemSeparatorComponent</code>	Componente JSX	Separador entre ítems
<code>ListEmptyComponent</code>	Componente JSX	JSX cuando <code>data</code> está vacío
<code>numColumns</code>	número entero	Renderizado en cuadrícula
<code>contentContainerStyle</code>	StyleProp	Padding interior del scroll

👉 ¡Enhorabuena! Ahora tienes las herramientas para crear listas de alto rendimiento con `FlatList` en cualquier proyecto React Native. 🚀